🎙️ SPEECH D'ENTRAÎNEMENT ORAL — BLOC 4 : CLINER-MLOPS (INDUSTRIALISATION & FINOPS)

Certification Architecte en Intelligence Artificielle (RNCP 41993 — Niveau 7)
Durée du Speech : 5 Minutes Chrono *(suivi de 10 min de Q&A)*
Alignement Livrables (`PRESENTATION_cliner-mlops.pptx`) : Synchronisé à 100 % au mot et au chiffre près avec ta dernière version de `PRESENTATION_cliner-mlops.pptx` *(6 slides)* et le dépôt GitHub `cliner-mlops`.
Posture : Tu es l'**Architecte MLOps & LLMOps** qui a conçu et automatisé l'intégralité du cycle de vie de **CliNER-MLOps** (Dataset CHIA ➔ Prétraitement RAG BioBERT ➔ QLoRA 4-bit Qwen2.5-7B ➔ Déploiement Hybride Render/AWS/Cloud Run + Terraform ➔ CI/CD GitHub Actions ➔ Drift Wasserstein & Evidently ➔ Continuous Training & Rollback S3 ➔ Auto-Kill FinOps `0,18 $`).

⏱️ RÉCAPITULATIF DU CHRONO (5 MINUTES — SLIDES 1 À 6)

Chrono	Slide	Titre Exact de la Diapositive (`PRESENTATION_cliner-mlops.pptx`)	Critère Officiel Validé
00:00 – 00:15	**Slide 1**	*CHRISTOPHER GILLERON — BLOC 4 FINAL PROJECT AI ARCHITECT*	Introduction Exécutive
00:15 – 01:10	**Slide 2**	*1. Enjeu Clinique, Dataset CHIA & Prétraitement RAG*	**1. Jeu de données & Prétraitement**
01:10 – 02:05	**Slide 3**	*2. Entraînement QLoRA 4-bit, Métriques CHIA & Serving*	**2. Entraînement, Hyperparamètres & Métriques**
02:05 – 03:10	**Slide 4**	*3. Architecture Cloud Hybride, Docker & CI/CD*	**3a. Déploiement Cloud & 3b. CI/CD**
03:10 – 04:20	**Slide 5**	*4. Drift Wasserstein, Continuous Training, Rollback & FinOps*	**3c. Monitoring, 3d. Réentraînement & 3e. Rollback**
04:20 – 05:00	**Slide 6**	*5. Démonstration Bout-en-Bout, API (`/docs`) & Bilan*	**3f. Documentation API & Scalabilité**

🗣️ TEXTE À LIRE À L'ORAL (STYLE PROMPTEUR FLUIDE)

🫀 SLIDE 1 — Introduction Exécutive *(⏱️ 00:00 – 00:15 | 15 sec)*

« Bonjour à tous. Pour ce Bloc 4 consacré à l'industrialisation MLOps, je vous présente **CliNER-MLOps** : un pipeline Cloud hybride qui extrait automatiquement les critères d'éligibilité des essais cliniques en **3 secondes**, détecte la dérive des données en production, et réentraîne un modèle **Qwen2.5-7B** sur AWS en mode *Auto-Kill FinOps*. »

- 👉 **[Passer à la Slide 2]**

🧬 SLIDE 2 — 1. Enjeu Clinique, Dataset CHIA & Prétraitement RAG *(⏱️ 00:15 – 01:10 | 55 sec)*

> « Sur la **Slide 2**, partons du problème médical : sur *ClinicalTrials.gov*, on compte plus de **450 000 essais cliniques** non standardisés sous forme de PDF denses de **50 à 150 pages**. Un médecin met **45 minutes** à analyser un seul essai, ce qui retarde le recrutement de **80 % des essais cliniques mondiaux** *(études Tufts CSDD et Sully et al.)*. Et envoyer 100 pages brutes à un LLM générique provoque des hallucinations (*Lost in the Middle*), plus de **4 minutes de latence** et un coût VRAM massif.
>
> Pour y répondre, j'ai construit un pipeline de données rigoureux :
>
> 1. **Benchmark CHIA (*Nature 2020*) :** j'ai sélectionné ce jeu de données médical de référence qui rassemble **1 000 études cliniques**, **12 409 critères** et plus de **41 000 entités** annotées manuellement par des médecins.
> 2. **Prétraitement Sans Fuite (*Zero Data Leakage*) :** j'ai converti les annotations BRAT au format **ChatML** en ciblant les **9 classes médicales** essentielles, avec une séparation hermétique de **800 études d'entraînement** et **200 études de test** (zéro donnée synthétique).
> 3. **Ingestion Duale & Filtrage RAG BioBERT :** les protocoles issus de l'API *ClinicalTrials.gov v2* ou de PDF *PyMuPDF* sont archivés sur **AWS S3**, découpés en chunks de **1 000 caractères** et vectorisés en **768 dimensions (12 ms)** par **BioBERT** dans **Supabase** `pgvector` pour isoler le **Top-5 chunks**, éliminant **98 % du bruit** avant l'appel au LLM. »

- 🛡️ *Raccord 100 % Slide 2 : Tous les chiffres (* `450 000+` *,* `50 à 150 pages` *,* `45 min` *,* `80 % Tufts CSDD` *,* `1 000 études / 12 409 critères / 41 000+ entités` *,* `800/200` *,* `1000 car. / 768d / 12 ms / Top-5 / -98 % bruit` *) sont dans l'ordre exact de ta Slide 2.*
- 👉 **[Passer à la Slide 3]**

🧠 SLIDE 3 — 2. Entraînement QLoRA 4-bit, Métriques CHIA & Serving *(⏱️ 01:10 – 02:05 | 55 sec)*

> « Sur la **Slide 3**, passons à l'entraînement et au serving du modèle :
>
> - **Choix de Qwen2.5-7B + QLoRA 4-bit (NF4) :** un fine-tuning complet d'un modèle 7B exigerait plus de **80 Go de VRAM**. Grâce à la quantification **4-bit NormalFloat**, le backbone figé tient dans seulement **5 Go de VRAM** sur une carte **Nvidia T4 à 0,56 $/h**.
> - **Hyperparamètres Optimisés :** j'ai fixé un **rang** `r=16`, un facteur `alpha=32`, un `dropout=0.05` et un `learning_rate=2e-4` sur les couches d'attention (`q, k, v, o`). Notre adaptateur LoRA ne pèse que **84 Mo au lieu de 15 Go** (**-99,4 % de poids**), ce qui permet de le versionner en **2 secondes sur AWS S3 (** `models_lora/vN/` **)**, et d'extraire les **9 classes** médicales selon un schéma JSON strict sans hallucination.
> - **Métriques CHIA :** en **20 minutes** de convergence, la Loss chute régulièrement de **1,450 à 0,284** dans MLflow. Évalué en **Exact Boundary Match au caractère près**, le modèle atteint **58,3 % de F1-Score strict** et **63,0 % de Précision** (très proche de l'accord inter-annotateurs humain de 65 à 70 %).
> - **Inférence GPU vLLM & Cache SQL :** en production, **vLLM (*PagedAttention*, 75 tok/s)** charge l'adaptateur à chaud via `LoRARequest` et répond en **moins de 3 secondes**. Et notre table de cache `clinical_ner_cache` sur Supabase, indexée sur la **clé** `NCT` **(** `doc_id` **)**, renvoie les essais déjà analysés en **0,01 seconde pour 0 €**. »

- 🛡️ *Raccord 100 % Slide 3 :* `5 Go VRAM vs > 80 Go`, `r=16, α=32, dropout=0.05, lr=2e-4`, `84 Mo vs 15 Go (-99,4 %)`, `9 classes`, `Loss 1.450 → 0.284`, `F1 58,3 % / Précision 63,0 %`, `vLLM 75 tok/s < 3s`, `Cache SQL Clé NCT 0,01s / 0 €`.
- 👉 **[Passer à la Slide 4]**

☁️ SLIDE 4 — 3. Architecture Cloud Hybride, Docker & CI/CD *(⏱️ 02:05 – 03:10 | 1 min 05)*

> « La **Slide 4** présente notre architecture Cloud hybride structurée en **4 blocs** :
>
> 1. **En haut à gauche — Le Front-End Streamlit sur Render (0 €) :** il propose 4 onglets aux cliniciens : recherche live sur l'API v2 et import PDF, *Summary Table* exportable en CSV/S3, visualisation des entités NER (badges et JSON), et Chatbot médical RAG.
> 2. **En haut à droite — Le Data Lake & Cache (Terraform + AWS S3 + Supabase) :** notre code **IaC Terraform** provisionne en moins de 30 secondes le bucket **AWS S3** (qui stocke les PDF bruts, le dataset CHIA et le registre des adaptateurs LoRA `v1, v2...` de 84 Mo), ainsi que la base **Supabase PostgreSQL** comprenant le **cache NER (clé `doc_id` NCT en 0,01 s)** et l'index vectoriel **HNSW 768d (12 ms)** avec purge automatique **RGPD à 7 jours (`pg_cron`)**.
> 3. **En bas à droite — Le Backend GPU & MLOps (FastAPI / vLLM / AWS EC2 T4) :** il assure 4 fonctions : le filtrage BioBERT (**Top-5 chunks pour le NER** et **Top-15 chunks pour le Chatbot RAG**), l'inférence **vLLM Qwen2.5-7B QLoRA 4-bit**, le monitoring du drift de **Wasserstein (`W₁ > 0,08`)** et la boucle d'entraînement **Auto-Kill Boto3**.
> 4. **En bas à gauche — La CI/CD, l'Observabilité & l'Audit RGPD :** à chaque `git push`, **GitHub Actions** exécute le linter `flake8`, **11 tests PyTest** et le build **Docker**. Notre serveur **MLflow** est hébergé sur **Google Cloud Run en Scale-to-Zero (`0 €`)**, et chaque action est tracée dans des journaux **XML conformes ISO 13485 et RGPD**. »

- 🛡️ *Raccord 100 % Slide 4 : Les 4 blocs correspondent exactement aux 4 boîtes et aux flèches de ta Slide 4.*
- 👉 **[Passer à la Slide 5]**

🔄 SLIDE 5 — 4. Drift Wasserstein, Continuous Training, Rollback & FinOps *(⏱️ 03:10 – 04:20 | 1 min 10)*

> « Sur la **Slide 5**, voyons comment notre boucle de **Continuous Training** et de **FinOps** fonctionne en autonomie :
>
> - **Monitoring du Drift Non Supervisé (`W₁`) & Rollback S3 :** en production, les nouveaux protocoles PDF ne sont pas annotés par des médecins ; on ne peut donc pas calculer un F1-Score en direct. Nous calculons la **distance de Wasserstein (`W₁`)** sur les embeddings BioBERT 768d entre le corpus CHIA et les nouveaux essais, couplée à un rapport HTML **Evidently AI** :
> - Si `W₁ ≤ 0,08`, le modèle est stable (**0 €**).
> - Si `W₁ > 0,08`, l'alerte déclenche le réentraînement automatique. Et si le F1-Score du nouvel adaptateur `vN` régresse sur le jeu de validation CHIA, le système effectue un **Rollback Zero-Downtime immédiat vers la version** `vN-1` stockée sur S3 via `LoRARequest`.
> - **Orchestration EC2 JIT & Double Ceinture Auto-Kill FinOps :** par défaut, notre instance GPU **AWS EC2** `g4dn.xlarge` **(Tesla T4)** est éteinte (**0,00 €/h**). Dès que `W₁ > 0,08`, **Boto3** réveille l'EC2 en **45 secondes**, exécute les **20 minutes** de QLoRA 4-bit, pousse l'adaptateur de **84 Mo** sur S3 et MLflow, puis une **double ceinture de sécurité** — bloc `finally:` **Python** + étape **CI/CD** `if:`

> `always()` — force l'extinction `StopInstances`. Résultat : **0,18 $ par session** au lieu de 400 $/mois (**-99,9 % d'économie**).
> - **Auditabilité ISO 13485 & Sécurité :** tous les événements génèrent un journal d'audit **XML horodaté ISO 8601 (`/logs/xml`)** et aucun secret n'est stocké en dur grâce à **Google Secret Manager** et **GitHub Secrets**. »

- 👉 **[Passer à la Slide 6]**

🖥️ SLIDE 6 — 5. Démonstration Bout-en-Bout, API (`/docs`) & Bilan *(⏱️ 04:20 – 05:00 | 40 sec)*

> « Enfin, sur la **Slide 6**, tout ce cycle est démontré de bout en bout et prêt pour la production :
>
> 1. **Côté Praticien :** recherche live sur *ClinicalTrials.gov v2* et tri instantané sur Render CPU à 0 €, puis extraction NER JSON en **3 secondes** (ou **0,01 s** via le cache SQL) et Chatbot médical RAG.
> 2. **Côté Développeur :** une API REST FastAPI documentée sous **Swagger (`/docs`)** exposant les 5 routes clés : `/process_text` , `/process_pdf` , `/chat_rag` , `/monitoring/drift-report` et `/logs/xml` .
> 3. **Côté MLOps :** l'exécution de notre orchestrateur `run_pipeline.py --threshold 0.08` , qui détecte une dérive réelle à `0,0949` , génère le rapport Evidently, trace le run dans MLflow sur Cloud Run et coupe l'instance EC2 pour **0,18 $**.
>
> Je vous remercie pour votre attention, et je suis à votre disposition pour vos questions ! »

🛡️ TOP 5 QUESTIONS RÉFLEXES DU JURY APRÈS CE SPEECH (RÉPONSES EN 25s)

1. **« Pourquoi utilisez-vous à la fois Terraform (`terraform/main.tf`) et Boto3 (`ec2_manager.py`) pour gérer AWS ? »**

   > *« C'est la séparation entre Provisioning structurel (IaC) et Orchestration Runtime : Terraform crée une seule fois l'infrastructure permanente (le bucket S3 versionné et chiffré AES-256, le schéma Supabase `pgvector` /HNSW/ `pg_cron` , et l'instance EC2 GPU avec son disque EBS de 50 Go pré-configuré avec CUDA et PyTorch). Ensuite, au quotidien, on ne fait jamais `terraform destroy` après un entraînement (ce qui détruirait le disque EBS de 50 Go !) : c'est notre script Python Boto3 (`ec2_manager.py`) qui allume et éteint électriquement la machine existante (`start_instances` / `stop_instances`) en 45 secondes ! »*

2. **« Pourquoi votre serveur MLflow sur Google Cloud Run coûte-t-il vraiment 0 € et comment ne perd-il pas ses données quand il s'éteint ? »**

   > *« Parce que sur Google Cloud Run configuré en `min_instances = 0` , le conteneur s'éteint complètement à zéro instance dès qu'aucun ingénieur ne consulte l'interface web (et les 2 millions de requêtes gratuites du Free Tier couvrent largement nos appels d'entraînement). Surtout, le conteneur est 100 % stateless : il ne stocke rien sur son disque local éphémère, mais utilise Supabase PostgreSQL comme* Backend Store *(métriques/paramètres) et AWS S3 comme* Artifact Store *(poids LoRA de 84 Mo). »*

3. **« Pourquoi utilisez-vous un Top-5 chunks pour le NER (`/process_text`), mais un Top-15 chunks pour le Chatbot RAG (`/chat_rag`) ? »**

> *« Parce que les deux tâches ont des objectifs différents : pour l'extraction NER structurée d'un protocole unique, les critères d'inclusion et d'exclusion tiennent dans 5 chunks de 1 000 caractères, ce qui maximise la précision JSON et garantit une latence de 3 secondes. À l'inverse, dans le Chatbot médical RAG, le médecin pose des questions transversales comparant plusieurs essais cliniques en base : remonter le Top-15 chunks offre une couverture multi-essais complète sans perte de rappel. »*

4. **« Que se passe-t-il si votre base Supabase PostgreSQL ou votre serveur GPU AWS tombe en panne en pleine journée ? »**

> *« J'ai implémenté un mode de résilience Zero-Downtime (Graceful Degradation) dans `api/main.py` : si Supabase ne répond pas en 4 secondes, l'API bascule automatiquement sur un dictionnaire vectoriel en mémoire RAM (`MEMORY_DOC_CHUNKS`) avec calcul cosinus NumPy local pour ne jamais interrompre le service. Et côté frontend Streamlit sur Render, si le GPU est éteint, le cache SQL et un extracteur déterministe de secours prennent le relais instantanément. »*

5. **« Comment avez-vous calibré le seuil de distance de Wasserstein à `0.08` ? »**

> *« Je l'ai calibré empiriquement par validation croisée sur nos embeddings BioBERT normalisés L2 : entre deux sous-échantillons aléatoires du même domaine clinique dans CHIA, la fluctuation naturelle (bruit statistique) de Wasserstein oscille entre `0.02` et `0.05`. En revanche, lorsqu'on injecte des protocoles d'une nouvelle spécialité médicale ou de thérapie génique récente, la distance bondit au-delà de `0.09` (`0.0949` dans notre test). Fixer le seuil à `0.08` évite les faux positifs coûteux tout en garantissant la détection de tout vrai changement de domaine. »*